

SOUTENANCE DEVSECOPS

Zero-Trust mTLS pour microservices

Une histoire en trois temps.

Sylvain Rougié

Soutenance DevSecOps

1
2
3

Trois temps, trois problèmes à résoudre

1

IDENTITÉ

Prouver qui appelle

Go + SPIRE + mTLS

2

MULTI-LANGAGE

Un service en Node

+ Envoy en sidecar

3

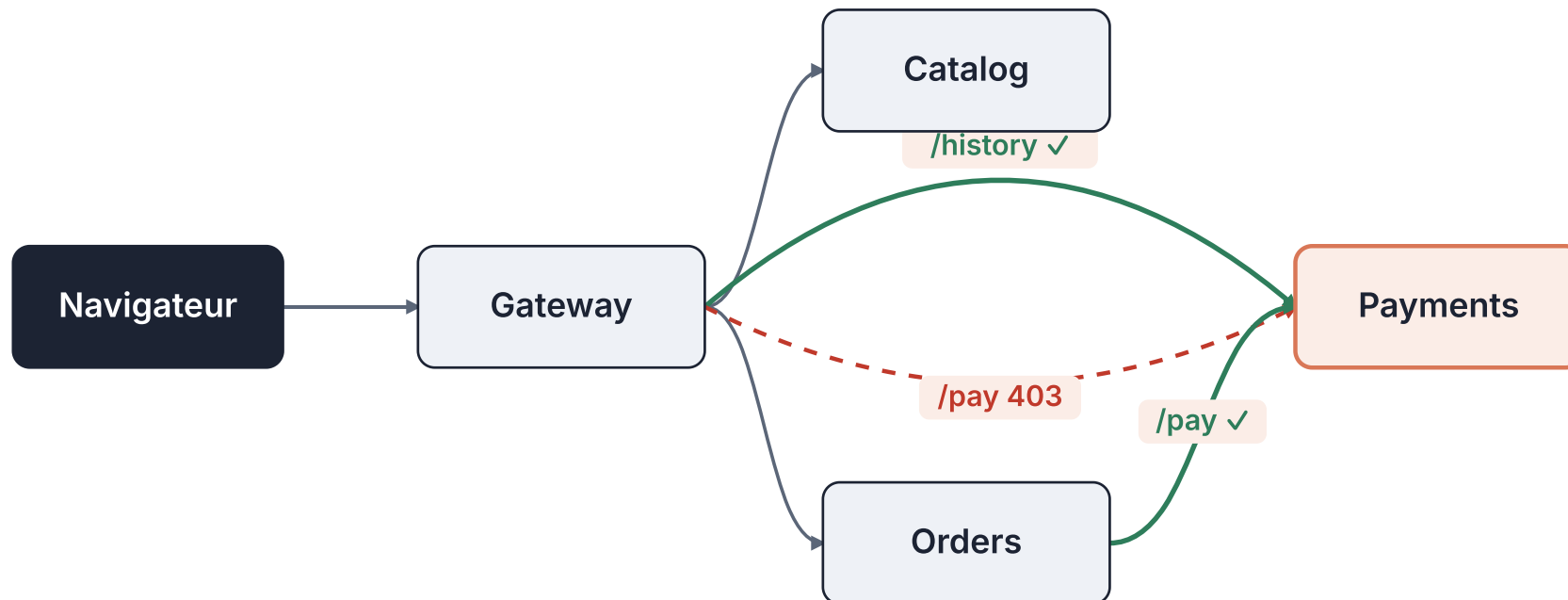
ÉCHELLE

Le câblage devient lourd

→ service mesh Istio

Une boutique en microservices, une règle à protéger

La règle : Gateway peut lire **/history**, mais seul Orders peut appeler **/pay**



| La stack, en un coup d'œil

Trois plans empilés — c'est la carte à garder en tête pour la suite

① LES WORKLOADS

5 services

- Gateway · Orders · Catalog · Payments (Go)
- Analytics (Node.js)
- Loadgen (trafic continu)

La règle `/pay → Orders` seul vit ici.

② LA SÉCURITÉ RUNTIME

Identité + mTLS

- v1 — SPIRE + mTLS dans le code Go
- v2 — service mesh **Istio**
- Observé dans **Kiali**

C'est l'histoire en 3 actes qui vient.

③ LA CHAÎNE & L'INFRA

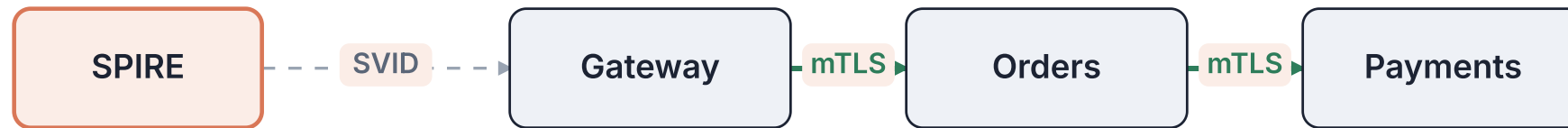
CI/CD + GitOps

- Checkov · Semgrep · Trivy
- Cosign → GHCR → ArgoCD
- Terraform + Ansible · k3s · SSH JIT

Tout est **reproductible from scratch**.

| Donner une identité à chaque service

SPIRE émet un certificat par workload · les services parlent en mTLS



Payments sait, de façon prouvée, que c'est Orders

| Pourquoi mTLS et pas une NetworkPolicy ?

COUCHE	OUTIL	IDENTITÉ BASÉE SUR	GRANULARITÉ	« /PAY VS /HISTORY » ?
L3 / L4	NetworkPolicy native	label → pod (contextuel)	pod + port	inexprimable
L7 réseau	Cilium (CNNP, eBPF)	label → pod, + HTTP	route / méthode	oui, mais identité réseau
L7 appli	mTLS + SPIFFE (mon choix)	clé privée / certificat	route / méthode	oui, par identité prouvée

La NetworkPolicy prouve d'où vient un paquet.
Le mTLS prouve **qui l'a signé**.

|Comment ça marche, concrètement

①

SPIRE émet

un X.509-SVID

②

Handshake mTLS

les deux pairs s'authentifient

③

Identité lue

dans le certificat

④

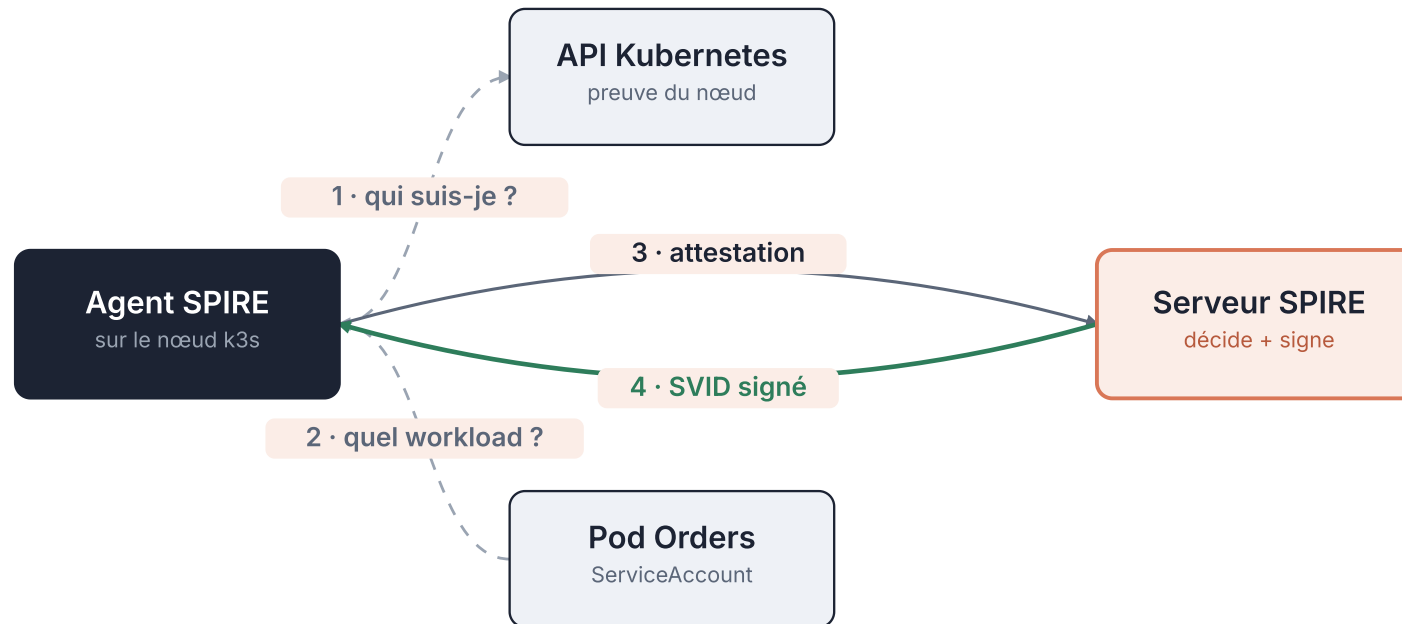
Décision

policy par route

L'identité est **prouvée pendant le handshake**, jamais déclarée dans un header.

Mais comment SPIRE sait que ce pod est Orders ?

Le SVID n'est pas donné sur parole — il est mérité par une double attestation



L'identité découle de **ce que le pod EST** (son ServiceAccount), pas d'un secret qu'on lui confie.

Pas de mot de passe à voler, pas de clé à distribuer.

|Le seul bout de code qui compte

```
// authentifié ≠ autorisé : on vérifie la policy
func handlePay(r *Request) {
    id := r.TLS.PeerCertificates[0].URIs[0]

    if !policy["/pay"].allows(id) {
        return Error(403) // refusé
    }
}
```

Non forgeable

Se faire passer pour Orders exigerait sa **clé privée**, renouvelée en continu par SPIRE.

Gateway a une identité valide... mais reste **refusée** sur `/pay`.

ATTENDEZ...

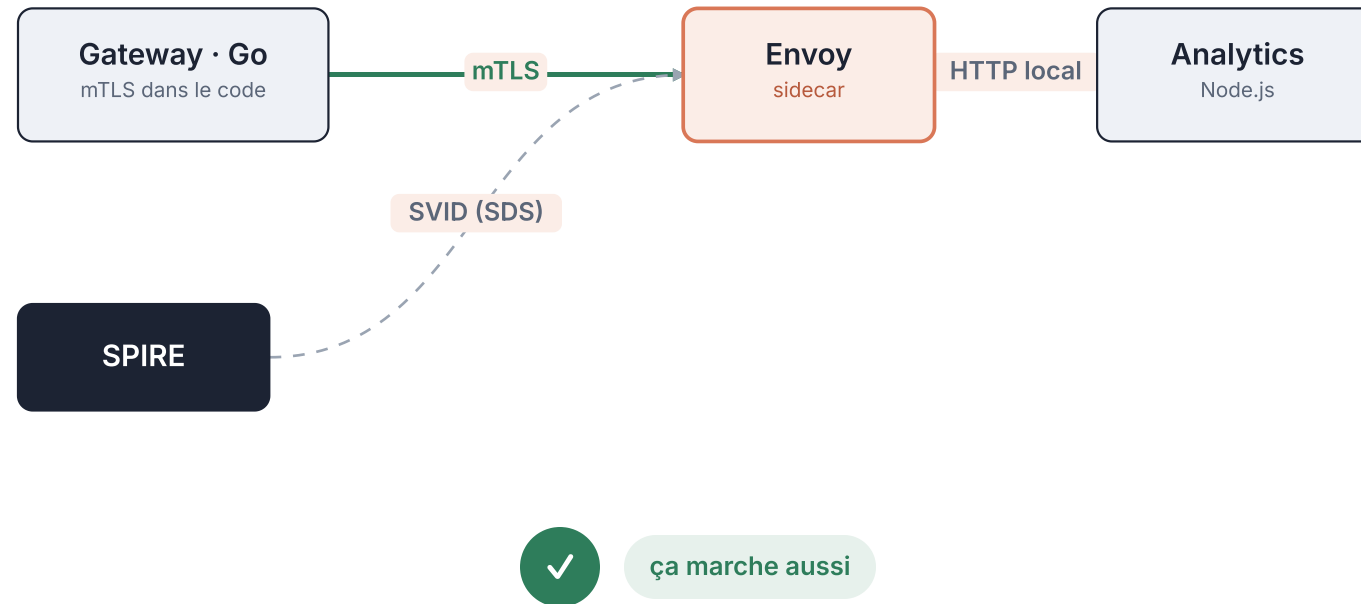
Et si on ajoute un service... dans un autre langage ?

Analytics sera écrit en Node.js. Pas de go-spiffe en Node — on réimplémente tout ?



Déléguer le mTLS à un proxy

Analytics reste du Node pur · Envoy en sidecar porte SPIFFE/mTLS



ATTENDEZ...

Vous voyez le problème ?
On recâble la même chose. Partout.

Sidecar à configurer à la main, mTLS répété service par service, policy éparpillée.



Trois douleurs qui ne passent pas à l'échelle



Répétition

La même plomberie mTLS recâblée
dans chaque service



Friction multi-langage

Chaque langage = une nouvelle
intégration à porter à la main

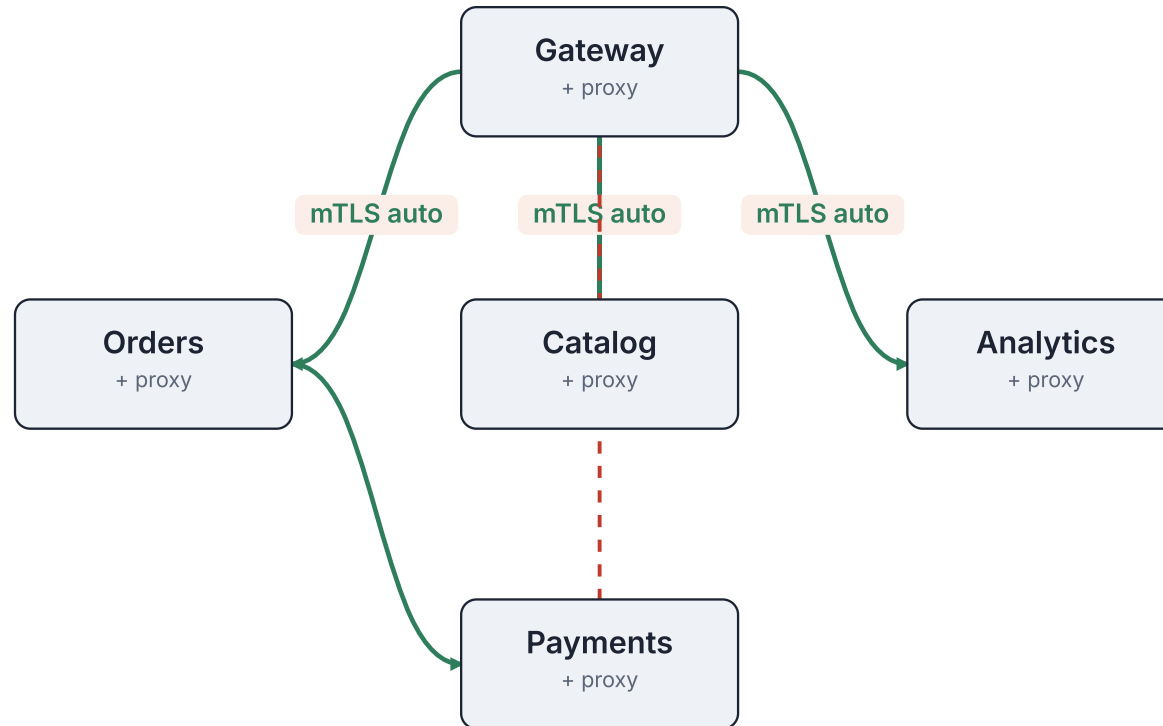


Policy dispersée

Changer une règle = modifier,
recompiler, redéployer

Le service mesh fait tout, partout

Istio injecte un proxy par pod · mTLS STRICT automatique, policy déclarative



Un seul mécanisme, **identique pour Go et Node** — et le 403 zero-trust, en config.

! Du coup, qui émet l'identité en v2 ?

La question piège : SPIRE disparaît, mais le mécanisme reste le même

V1 — L'APP FAIT LE TRAVAIL

- **SPIRE** émet le SVID
- Le code Go **lit le certificat** du pair
- L'app décide elle-même

Identité **dans le code**.

V2 — LE PROXY FAIT LE TRAVAIL

- **istiod** (sa CA) émet le certificat
- Le proxy termine le mTLS et injecte l'identité dans `X-Forwarded-Client-Cert`
- L'**AuthorizationPolicy** décide, hors du code

Identité **dans la plateforme**.

Même principe — **une identité prouvée au handshake**. Seul **l'émetteur** change : l'app cède la place au mesh.

La sécurité change de couche

AVANT — DANS LE CODE

- mTLS écrit en Go, service par service
- Envoy configuré à la main pour Node
- policy d'autorisation compilée dans l'app
- à refaire pour chaque service

APRÈS — DANS LA PLATEFORME

- mTLS automatique (1 ligne de config)
- même proxy injecté pour Go ET Node
- AuthorizationPolicy déclaratives (YAML)
- gratuit pour chaque nouveau service

La sécurité a changé de couche

La sécurité commence dans la CI/CD

Chaque merge franchit deux temps : on porte (scans bloquants), puis on livre (GitOps)

① Porter — bloquant

Checkov

+ tfint · IaC

Semgrep

SAST · code Go

go vet + build

4 services

tout en parallèle, puis ↓

Trivy

scan CVE HIGH/CRITICAL — bloquant

Un seul échec → **exit ≠ 0** → le build s'arrête. C'est une porte, pas un avertissement.

↓ images vertes

② Livrer — GitOps

Cosign

signe (keyless)

→

GHCR

image : sha

→

deploy/prod

tag épinglé

→

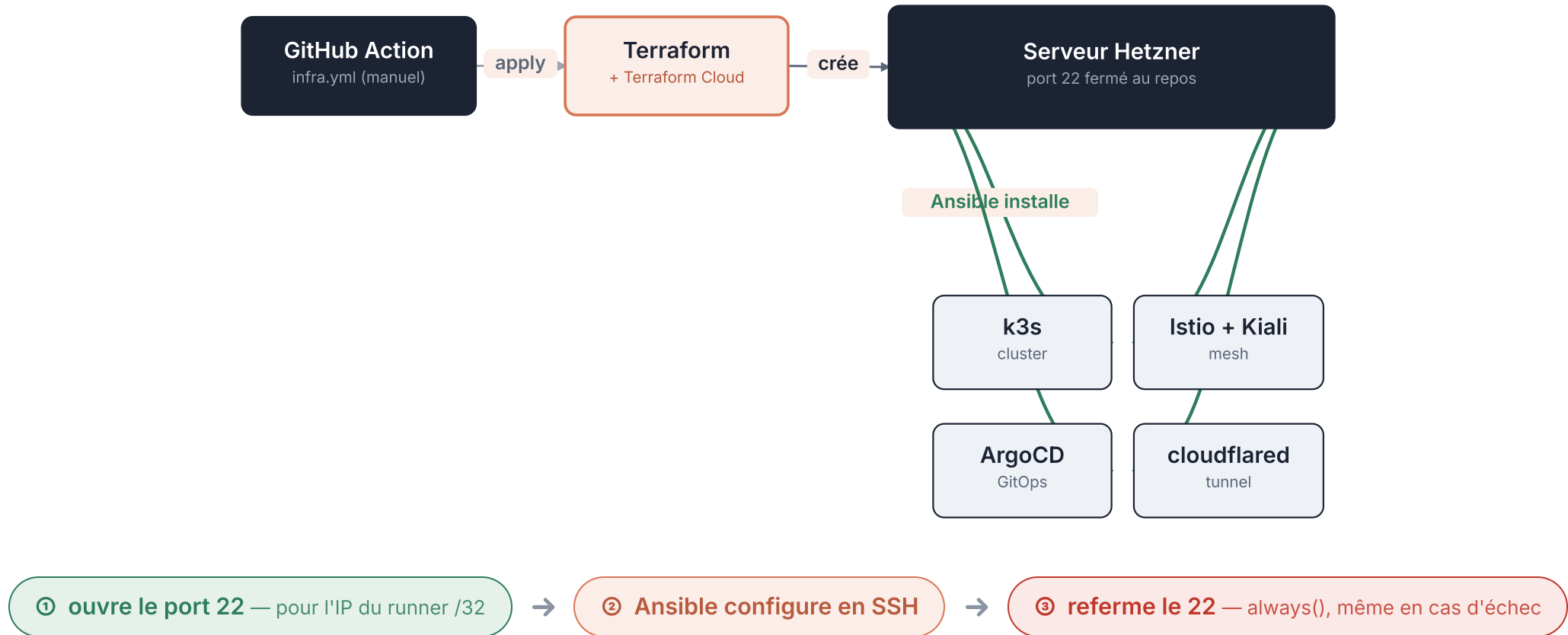
ArgoCD

sync → k3s

Pas de **latest** en prod. **Git fait foi** — un rollback = restaurer un commit.

| Une commande monte tout — et referme SSH derrière elle

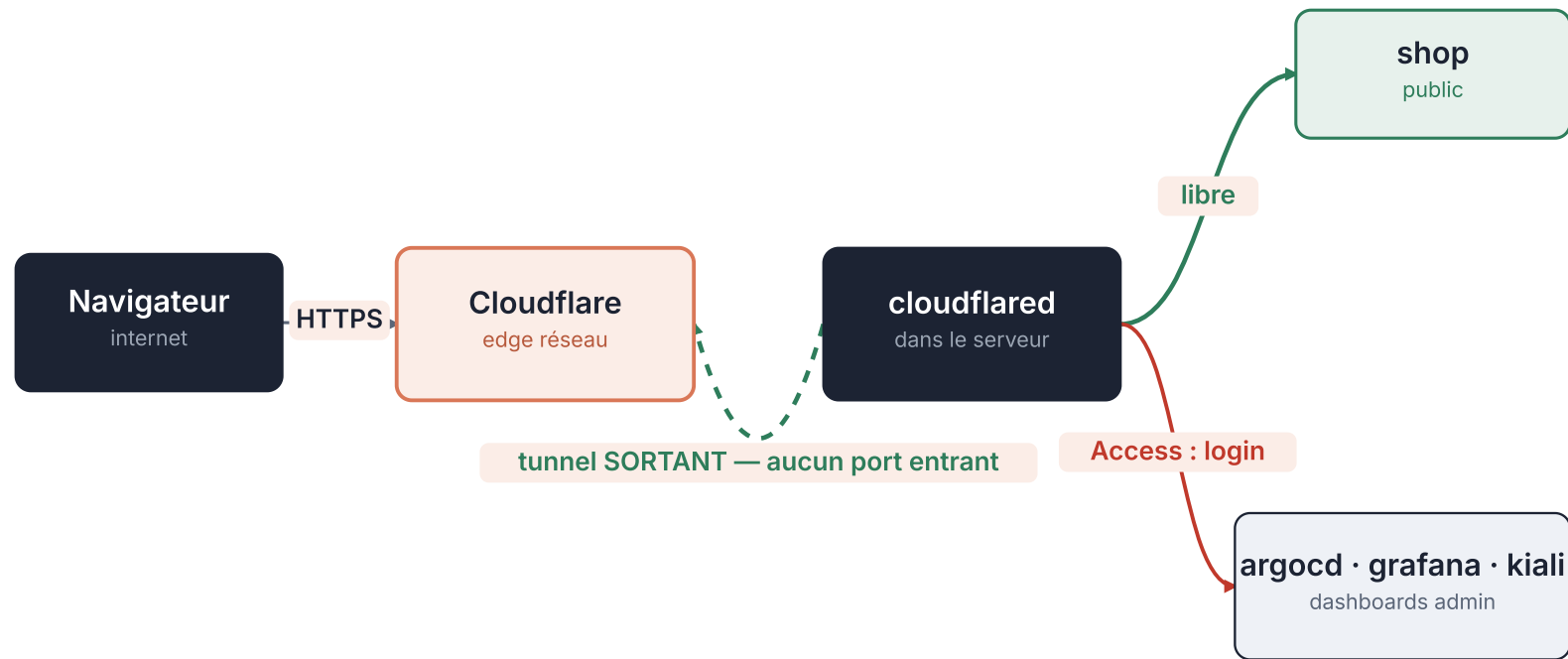
terraform crée le serveur · SSH s'ouvre le temps qu'Ansible pose le cluster · puis se referme



Hors de cette fenêtre, aucun port SSH ouvert : un scan de l'IP ne trouve rien.

| Zéro port entrant : le tunnel Cloudflare

Le serveur n'ouvre aucun port web · c'est cloudflared qui sort vers Cloudflare · les dashboards exigent un login



↑ **tunnel sortant** — le serveur n'expose **aucun port web** : rien à scanner

Access — login email vérifié à l'**edge**, avant même d'atteindre le cluster

Ce que cette histoire démontre

Chaque brique répond à un **problème réel**.
La sécurité a migré du **code** vers la **plateforme**.

01 Identité prouvée

SPIRE / SPIFFE · mTLS

02 Zero-trust visible

403 observable dans Kiali

03 Reproductible & signé

IaC · GitOps · supply chain

Merci.

Du code à la plateforme — une histoire zero-trust en trois temps

Sylvain Rougié